Primer módulo Python que consume un servicio externo (Bloque 5) / Construye tu módulo Python de consumo de servicios [Actividad práctica]

✓ **Hecho:** Ver

📘 **UNIDAD U1.3** · Python aplicado para integración con IA

Construye tu módulo Python de consumo de servicios

Ya viste la demostración: un módulo simple que consume un servicio, procesa la respuesta, y luego un error de red provocado a propósito para mostrar por qué el manejo de errores importa.

Ahora te toca a ti construir tu propia primera versión de ese módulo, la primera pieza de código real de tu proyecto en este programa. Esta versión va a ser deliberadamente simple, sin manejo de excepciones todavía: eso llega en el Bloque 7.

> El objetivo de hoy es que la función funcione correctamente en el camino esperado, antes de blindarla contra lo inesperado.

Las dos librerías que vas a usar, "requests" y "python-dotenv", ya deberían estar instaladas en tu entorno virtual desde la Unidad 1.2. Si por algún motivo no lo están (por ejemplo, si estás trabajando desde un entorno virtual nuevo), instálalas primero con tu entorno virtual activo:

```
pip install requests python-dotenv
```

🛠️ Qué vas a construir

➡️ Dentro de la carpeta **"src"** de tu repositorio, vas a crear un archivo nuevo con una función: recibe los datos necesarios para identificar qué se está pidiendo, envía una petición a un servicio externo, convierte la respuesta de **JSON** a una estructura de Python, y devuelve el dato específico que interesa, ya extraído.

➡️ Devuelve ese resultado con "return", no lo imprimas directamente dentro de la función: imprimir es responsabilidad de quien use la función después, no de la función misma.

Ese mismo principio de responsabilidad única lo viste en la primera lección de este bloque, y es uno de los cuatro criterios de la rúbrica de calidad de código.

Esta lección usa como referencia la **API** de **Google Gemini**, la misma que ya conectaste desde Postman o Thunder Client en el Bloque 4: tu función va a enviarle una pregunta y va a devolver el texto que el modelo genere como respuesta.

Si tu proveedor de modelo es otro (**Groq**, por ejemplo), el patrón es idéntico; solo cambian la URL del endpoint, el nombre del encabezado o parámetro de autenticación, y el nombre exacto de los campos dentro de la respuesta.

⚙️ Construye la función, paso a paso

1 **Crea el archivo dentro de "src".** En **VS Code**: clic derecho sobre la carpeta **"src"** en el explorador de archivos, "Nuevo archivo", y dale un nombre descriptivo (por ejemplo, **"consumir_gemini.py"**).

2 **Importa lo que necesitas:** la librería que hace las peticiones, y la que carga tu **".env"**.

3 **Carga tu archivo ".env"** al iniciar el programa.

4 **Define la función y el parámetro que necesita:** la pregunta o instrucción que le vas a enviar al modelo.

5 **Dentro de la función, lee tu credencial desde la variable de entorno,** arma la **URL** del endpoint con el nombre de tu modelo, y arma el cuerpo de la petición con esa pregunta.

6 **Envía la petición** y convierte la respuesta de **JSON** a una estructura de Python.

7 **Extrae el texto generado y devuélvelo con "return".** El camino exacto dentro de la respuesta (qué nombres de campo hay que recorrer) depende de tu proveedor; para Gemini, el texto generado queda dentro de **"candidates"**, después **"content"**, después **"parts"**.

Puesta junto, paso por paso, tu función se ve así:

```
import os
import requests
from dotenv import load_dotenv

 load_dotenv()

 def obtener_dato_de_prueba(pregunta):
     api_key = os.getenv("GEMINI_API_KEY")
     modelo = "gemini-3.6-flash"
     url = f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateConte
     body = {
        "contents": [
            {"parts": [{"text": pregunta}]}
        ]
     }
     respuesta = requests.post(url, json=body)
     datos = respuesta.json()
     return datos["candidates"][0]["content"]["parts"][0]["text"]
```

El nombre del modelo ("gemini-3.8-flash") es el que confirmaste que funciona en tu propia cuenta al probarlo desde Postman o Thunder Client en el Bloque 4; si tu proveedor es distinto, la URL, el nombre del

parámetro de la llave, y los nombres de campo dentro de "datos" van a ser diferentes. Revisa siempre la documentación del proveedor que elijas, y nunca escribas tu credencial directamente en el código: siempre debe venir de "os.getenv".

8 **Prueba la función**, todavía dentro del mismo archivo. Debajo de la función, agrega una prueba que imprima el resultado:

```
print(obtener_dato_de_prueba("Confirma en una frase corta que esta conexión fu
```

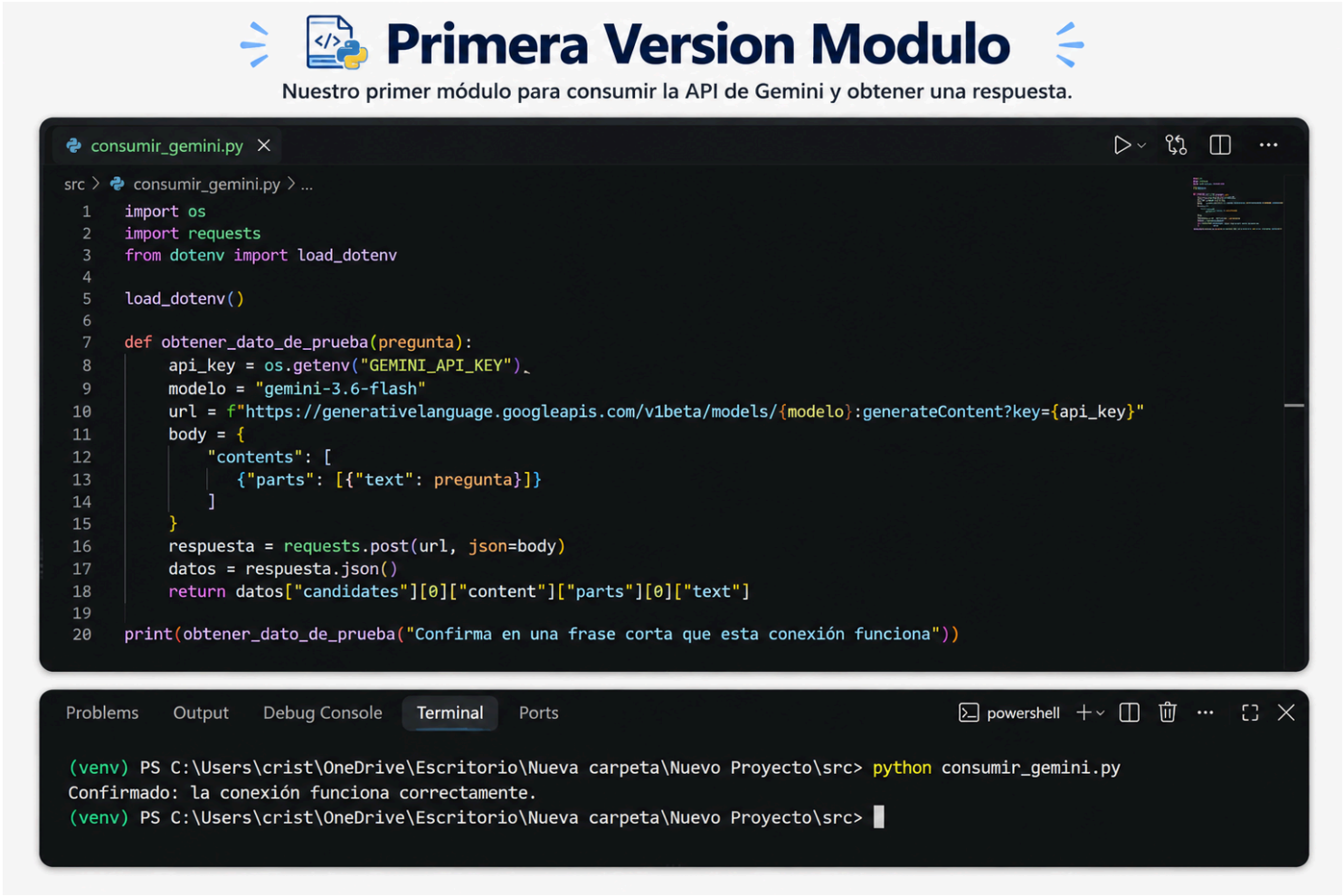
Como el modelo genera texto en cada ejecución, la respuesta exacta va a variar (no es u Para esta pregunta en particular, vas a ver algo parecido a esto en tu consola:

```
Sí, esta conexión funciona correctamente.
```

Y ejecuta el archivo completo desde la terminal, desde la raíz de tu proyecto:

```
python src/consumir_gemini.py
```

En VS Code también puedes ejecutar el archivo con el botón de ejecutar (el triángulo) e superior derecha del editor, en vez de escribir el comando a mano.



> **Si ves un error aquí**: la causa más común es una variable de entorno que no coincide. Si tu archivo ".env" no está en la raíz de tu proyecto, no se llama exactamente ".env", o el nombre de la variable no es exactamente **"GEMINI_API_KEY"**, "os.getenv("**GEMINI_API_KEY**")" no lanza ningún error ahí: simplemente devuelve "**None**" en silencio. El problema aparece un poco más adelante: la petición sale con una credencial inválida, la **API** responde con un error de autenticación en vez de tu respuesta, y tu código se rompe con "**KeyError: 'candidates**'" al intentar leer un campo que esa respuesta de error no tiene. Si ves ese mensaje, revisa primero tu ".env", no tu código.

🌐 Por qué usar directamente la API real, y no un servicio de prueba genérico

A diferencia de otros ejercicios de esta unidad, aquí conviene ir directo al proveedor real en vez de pasar primero por un servicio de prueba público: ya hiciste todo el trabajo previo para esto (conseguiste tu credencial, la protegiste en tu **".env"**, y confirmaste que funciona desde un cliente **HTTP** en el **Bloque 4**), así que usarla ahora no agrega fricción nueva, y sí te deja ver, desde esta primera versión, la estructura

real de la respuesta que vas a seguir procesando durante el resto de la unidad.

> 🎯 Eso sí: el **objetivo de esta lección sigue siendo el mismo patrón que si usaras un servicio de prueba cualquiera**: una función que recibe lo necesario, hace la petición, y devuelve el dato ya extraído.

Que el servicio de hoy sea el real no cambia ese patrón; lo vas a reconocer exactamente igual cuando más adelante uses esta misma estructura con cualquier otra **API**.

⚠️ Qué falta todavía, y por qué está bien que falte

Si pruebas esta función con una pregunta que provoque un error (por ejemplo, apagando tu conexión, o con un nombre de modelo que no existe), tu función se va a romper igual que se rompió en la demostración: sin ningún control.

> ☝️ Eso es esperado, **no un error tuyo**: el manejo de esos casos llega recién en el Bloque 7.

Tampoco te preocupes todavía por dividir esta lógica en varias funciones más pequeñas; ese es exactamente el trabajo del Bloque 6. Hoy el único objetivo es que, en el camino esperado (una pregunta válida, una respuesta exitosa), tu función funcione de principio a fin.

9

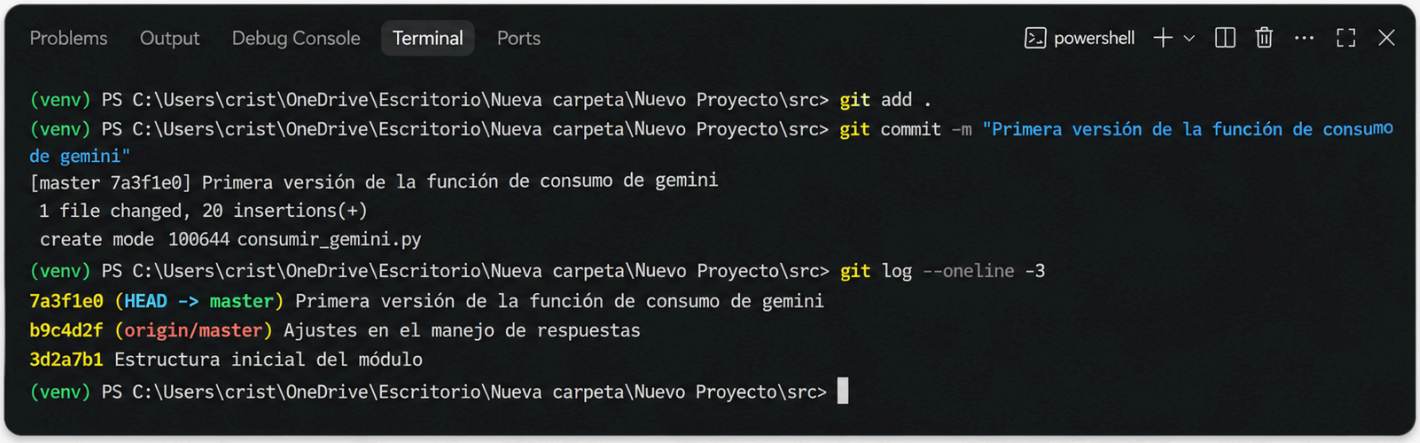
Guarda este avance con un commit:

```
git add .
git commit -m "Primera versión de la función de consumo de servicio"
```

En VS Code puedes hacer lo mismo desde el panel de control de código fuente, escribiendo el mismo mensaje en el campo de texto antes de confirmar el commit.

Historial Commit Modulo

Registro de los cambios realizados en nuestro módulo a lo largo del tiempo.



COMANDO
Acciones ejecutadas en Git.

COMMIT
Snapshot de cambios con un mensaje descriptivo.

HASH (ID)
Identificador único de cada commit.

RAMA
Indica la rama actual (master).

ORIGEN
Referencia al remoto (origin/master).

Importante: Hacer commits frecuentes con mensajes claros te ayuda a mantener un historial organizado y a identificar fácilmente cada cambio.

Siguiente etapa

Bloque 6 · De un script monolítico a funciones reutilizables

Con esta primera versión funcionando y ya en tu historial de commits, das el primer paso real de esta unidad. En el **Bloque 6** vas a tomar esta misma lógica y reorganizarla en funciones más pequeñas y reutilizables, el siguiente tramo del camino hacia el módulo completo que vas a entregar al cierre de la **Unidad U1.3**.

Última modificación: jueves, 24 de septiembre de 2026, 12:46